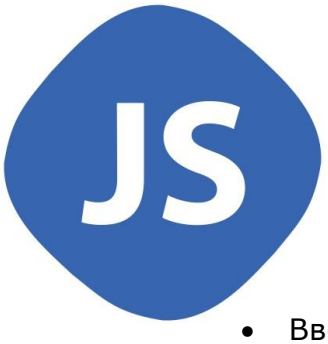


Объект Location

- Введение
- Свойства объекта
- Методы объекта
- Передача данных в строке запроса
- Практика построения навигационного меню

Введение

Универсальный идентификатор ресурсов (URL)

URL (унифицированный определитель местонахождения ресурса) - система обозначений для однозначной идентификации компьютера, каталога или файла в глобальной информационной сети Интернет.

В систему **URL** заложены следующие принципы:

- **Расширяемость** - новые адресные схемы должны легко вписываться в существующий синтаксис URL; расширяемость достигается за счет выбора определенного порядка интерпретации адресов, который базируется на понятии "адресная схема". Идентификатор схемы стоит перед остатком адреса, отделен от него двоеточием и определяет порядок интерпретации остатка.
- **Полнота** - по возможности любая из существовавших схем должна описываться посредством URL.
- **Читаемость** - адрес должен легко пониматься человеком, что вообще характерно для технологии WWW, - документы вместе с ссылками могут разрабатываться в обычном текстовом редакторе.

WWW — сокращение от World Wide Web, что переводится как "всемирная паутина", в современном русском языке обычно называется вебом (веб-разработчик, вебмастер и т.д).





Для полноценной работы, кроме URL необходимо знать еще несколько близких по смысловой нагрузке понятий.

URI, URL, URN

URI - Uniform Resource Identifier (унифицированный идентификатор ресурса). **Имя** и **адрес** ресурса в сети, включает в себя URL и URN.

> **URI** (**Uniform Resource Identifier**) – это строка символов, которая используется для идентификации какого-либо ресурса по его адресу или по его имени, либо по тому и тому вместе.

URL - Uniform Resource Locator (унифицированный определитель местонахождения ресурса). **Адрес** ресурса в сети, определяет местонахождение и способ обращения к нему.

> **URL** (**Uniform Resource Locator**) – это строка символов, которая используется для идентификации какого-либо ресурса, но только по его адресу, по его местоположению.

URN - Unifrorm Resource Name (унифицированное имя ресурса). **Имя** ресурса в сети, определяет только название ресурса, но не говорит как к нему подключиться.

> **URN** (**Uniform Resource Name**) – это строка символов, которая используется для идентификации какого-либо ресурса, но только по его имени.

> ***Если совсем кратко***.
>
> - **URN** – имя ресурса.

- **URL** – адрес ресурса.
- **URI** – имя и адрес ресурса (URI = URL + URN)

Пример:

- **URI** – https://mydomain.com/catalog/products.html
- **URL** - https://mydomain.com/
- **URN** - /catalog/products.html

В несколько упрощенном случае **формат адреса URL** выглядит следующим образом:

http: //js/ target.php?course=javascript&topic=bom&lesson=location#encode

origin – общая схема запроса **http://js**

protocol – протокол (включая двоеточие) **http:**

hostname – домен **js**

pathname – путь к ресурсу **target.php**

search – строка запроса - **?course=javascript&topic=bom&lesson=location**

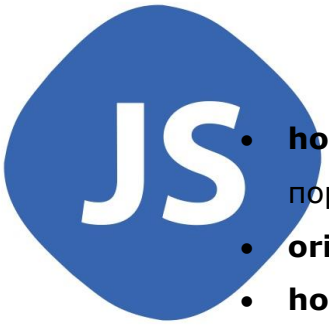
hash – идентификатор фрагмента **#encode**

Свойства объекта

Объект **location** содержит информацию о расположении текущей веб-страницы: **URL**, информацию о **сервере**, номер **порта**, **протокол**.

Эту информацию можно получить с помощью свойств объекта **location**:

- **protocol** – протокол (включая двоеточие), например, http: или https:



- **host** – хост, например, localhost.com. Если адрес URL содержит номер порта, то порт также входит в хост, например, localhost.com:8080;
- **origin** – общая схема запроса;
- **hostname** – домен, аналогичен хосту, только не включает порт, например, localhost.com;
- **href** – полный адрес URL веб-страницы;
- **pathname** – путь к ресурсу - та часть адреса, которая идет после хоста после слеша /;
- **search** – строка запроса - та часть адреса, которая идет после знака вопроса ? (при его налии);
- **hash** – идентификатор фрагмента - та часть адреса, которая идет после символа решетки # (при его наличии);
- **port** – порт, используемый ресурсом;
- **username** – имя пользователя, которое указано в адресе. Например, в адресе "https://john:qwerty@localhost.com" это подстрока "john";
- **password** – пароль, который указан в адресе. Например, в адресе "https://john:qwerty@localhost.com" это подстрока "qwerty".

В текущем уроке поработаем с адресной строкой браузера, а также разберем принцип построения панелей навигационных ссылок клиентским приложением. Покатили.

Разместите файл на локальном домене **http://js** или измените адрес ссылки в демонстрационном примере **example_1**. Для отслеживания значений свойств **location** переход на другую страницу при клике на ссылку не должен быть выполнен.

example_1.

```
<a
href="http://js/example_1.html?course=javascript&topic=bom&lesso
n=location#encode-search">
    Переход на страницу с передачей дополнительных параметров
</a><hr>
<script>
```

```
    // важные свойства объекта location
    document.write ("protocol: ", location.protocol, "<br>");
    document.write ("host: ", location.host, "<br>");
    document.write ("origin: ", location.origin, "<br>");
    document.write ("hostname: ", location.hostname, "<br>");
    document.write ("href: ", location.href, "<br>");
    document.write ("pathname: ", location.pathname, "<br>");
    document.write ("search: ", location.search, "<br>");
    document.write ("hash: ", location.hash, "<br>");
</script>
```

Идентификатор фрагмента (hash)

Идентификатор фрагмента (hash) – один из самых малозаметных и недостаточно часто используемых элементов строки запроса (к сожалению).

В демонстрационных примерах **example_2**-**example_5** тестируем возможности **идентификатора** совместно с **навигационными** ссылками страницы.

example_2.

```
<div id="menu">
    <a href="#article-1">Практические советы по улучшению
    юзабилити лендингов</a><br>
    <a href="#article-2">Как выбрать дизайн для лендинга и не
    пожалеть?</a><br>
    <a href="#article-3">Как сделать портфолио дизайнера:
    разбираем на примерах и даём советы начинающим</a><br>
    <a href="#article-4">Типографика: основы и правила для
    создания понятного и эффективного сайта</a><br>
    <a href="#article-5">Как связаны дизайн и конверсия сайта:
```

```
        неожиданные открытия</a><br>
        <a href="#article-6">Как сделать прототип лендинга</a><br>
        <hr>
</div><!-- end div#menu -->
<div id="content" style="width:75%; margin:0 auto">
    <div id="article-1">
        <h2>Практические советы по улучшению юзабилити
        лендингов</h2>
        <h3>Что такое юзабилити лендинга</h3>
        <p>
        Юзабилити и юзабилити лендинга – это не одно и то же.
        Юзабилити – это общий термин, который описывает удобство
        использования веб-ресурсов. Чаще всего это какие-то
        крупные интернет-порталы интернет-магазины с множеством
        различных функций.
        </P>
        <p>
        Юзабилити лендинга более специфично, фокусируется на
        создании удобного пользовательского опыта на лендинге.
        Лично я еще и применяю маркетинг.
        <P>
        Лендинги чаще всего разработаны для достижения той или
        иной маркетинговой цели, например, получить контакт
        клиента или продать товар.
        </p>
        <h3>Топ-4 ошибки юзабилити сайта</h3>
        <p>
        На самом деле, этих ошибок намного больше, я отобрала
        самые распространенные ошибки в дизайне, которые могут
```

```
            снизить эффективность лендинга.
            </p>
            <img src="images/2692-1701253136.jpg" width="500px">
        </div>
        <!-- ... -->
</div><!-- end div#content -->
```

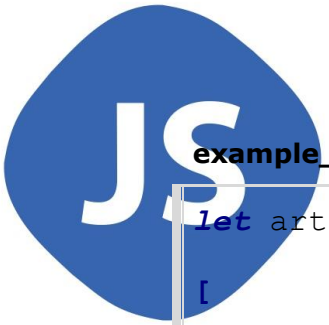
Перенесем **заголовки** статей в отдельный подключаемый массив (пока только заголовки).

example_3.

```
<script src="articles.js"></script>
<div id="menu"><hr></div>
<div id="content" style="width:75%; margin:0 auto">
    <div id="article-1">
        <h2>Практические советы по улучшению юзабилити
        лендингов</h2>
        <h3>Что такое юзабилити лендинга</h3>
        <p>
        Юзабилити и юзабилити лендинга – это не одно и то же.
        Юзабилити – это общий термин, который описывает удобство
        использования веб-ресурсов. Чаще всего это какие-то
        крупные интернет-порталы интернет-магазины с множеством
        различных функций.
        </P>
        <p>
        Юзабилити лендинга более специфично, фокусируется на
        создании удобного пользовательского опыта на лендинге.
        Лично я еще и применяю маркетинг.
```

```
        <P>
        Лендинги чаще всего разработаны для достижения той или
        иной маркетинговой цели, например, получить контакт
        клиента или продать товар.
        </p>
        <h3>Топ-4 ошибки юзабилити сайта</h3>
        <p>
        На самом деле, этих ошибок намного больше, я отобрала
        самые распространенные ошибки в дизайне, которые могут
        снизить эффективность лендинга.
        </p>
        <img src="images/2692-1701253136.jpg" width="500px">
    </div>
    <!-- ... -->
</div><!-- end div#content -->
<script>
    // формируем меню динамически
    let menu = '';
    for (article of articles) {
        menu += `
            <a href="#article-
            ${article.id}">${article.title}</a><br>
    `;
    }
    document.getElementById("menu").insertAdjacentHTML("afterbegi
    n", menu);
</script>
```

Массив заголовков статей представлен в файле **articles.js**.



example_3. **articles.js**

```
let articles =
[

    {

        id: 1,

        title: "Практические советы по улучшению юзабилити
        лендингов"

    },

    // ...
]
```

В демонстрационном примере **example_4** перенесем в подключаемый массив **контент** статей.

example_4.

```
<script src="articles.js"></script>

<div id="menu"><hr></div>

<div id="content" style="width:75%; margin:0 auto"></div>

<script>

    // начальная инициализация переменных

    let menu = '';

    let content = '';

    for (article of articles) {

        // контент меню

        menu += `

            <a href="#article-
            ${article.id}">${article.title}</a><br>

        `;
```

```
        // контент страницы
        content += `
            <div id="article-${article.id}">
                <h2>${article.title}</h2>
                ${article.content}
                <img src="images/${article.img}" width="500px">
            </div>
        `
    }
    // вставляем контент в браузер
    document.getElementById("menu").insertAdjacentHTML("afterbegi
    n", menu);
    document.getElementById("content").innerHTML = content;
</script>
```

Массив статей представлен в файле **articles.js**.

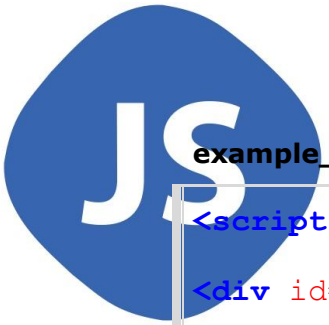
example_4. **articles.js**

```
let articles =
[
    {
    id: 1,
    title: "Практические советы по улучшению юзабилити
    лендингов",
    content: `<h3>Что такое юзабилити лендинга</h3>
    <p>
    Юзабилити и юзабилити лендинга – это не одно и то же.
    Юзабилити – это общий термин, который описывает удобство
```

```
    использования веб-ресурсов. Чаще всего это какие-то крупные
    интернет-порталы интернет-магазины с множеством различных
    функций.
    </P>
    <p>
    Юзабилити лендинга более специфично, фокусируется на создании
    удобного пользовательского опыта на лендинге. Лично я еще и
    применяю маркетинг.
    <P>
    Лендинги чаще всего разработаны для достижения той или иной
    маркетинговой цели, например, получить контакт клиента или
    продать товар.
    </p>
    <h3>Топ-4 ошибки юзабилити сайта</h3>
    <p>
    На самом деле, этих ошибок намного больше, я отобрала самые
    распространенные ошибки в дизайне, которые могут снизить
    эффективность лендинга.
    </p>`,
    img: "2692-1701253136.jpg"
    },
    // ...
]
```

В следующем примере установим ссылки для пунктов меню **динамически** (после вывода ссылки в браузер).

Динамически изменяемый (добавляемый) адрес ссылки (допустим термин – перелинковка) – действие, достаточно часто применяемое в веб-разработке.



example_5.

```
<script src="articles.js"></script>
<div id="menu"><hr></div>
<div id="content" style="width:75%; margin:0 auto"></div>
<script>
    // начальная инициализация переменных
    let menu = '';
    let content = '';
    for (article of articles) {
        // контент меню
        menu += `
            <a href="#article-
            ${article.id}">${article.title}</a><br>
        `;
        // контент страницы
        content += `
            <div id="article-${article.id}">
                <h2>${article.title}</h2>
                ${article.content}
                <img src="images/${article.img}" width="500px">
            </div>
        `
    }
    // вставляем контент в браузер
    document.getElementById("menu").insertAdjacentHTML("afterbegi
    n", menu);
```



```
    document.getElementById("content").innerHTML = content;
    // динамическое формирование ссылок меню
    // выбираем ссылки
    links = document.querySelectorAll("div#menu a");
    // для каждой ссылки меню установим идентификатор фрагмента
    links.forEach ( link => {
    link.setAttribute("href", "#article-" + link.dataset.article)
    } )
</script>
```

Массив файла статей **articles.js** без изменений.

При работе с идентификатором фрагмента очень важно понимать следующее (особенно для backend-разработчика) ...

> ***Важно***. **Идентификатор фрагмента на сервер не отправляется**. Получить доступ на сервере к нему нельзя.

Здесь важно понимать: то, что мы видим в адресной строке браузера и то, что браузер отправляет серверу – совсем не одно и тоже. Столкнемся с этим еще раз в разделе **Передача данных в строке запроса** чуть ниже по тексту урока.

Продемонстрирую сказанное на примере **example_6**. Пример использует **серверные** технологии программирования.

example_6.

```
<a
href="/target.php?course=javascript&topic=bom&lesson=location#da
ta-json">
    Переход к файлу target.php
```

```
</a>
```

Код целевого файла target.php.

example_6. target.php

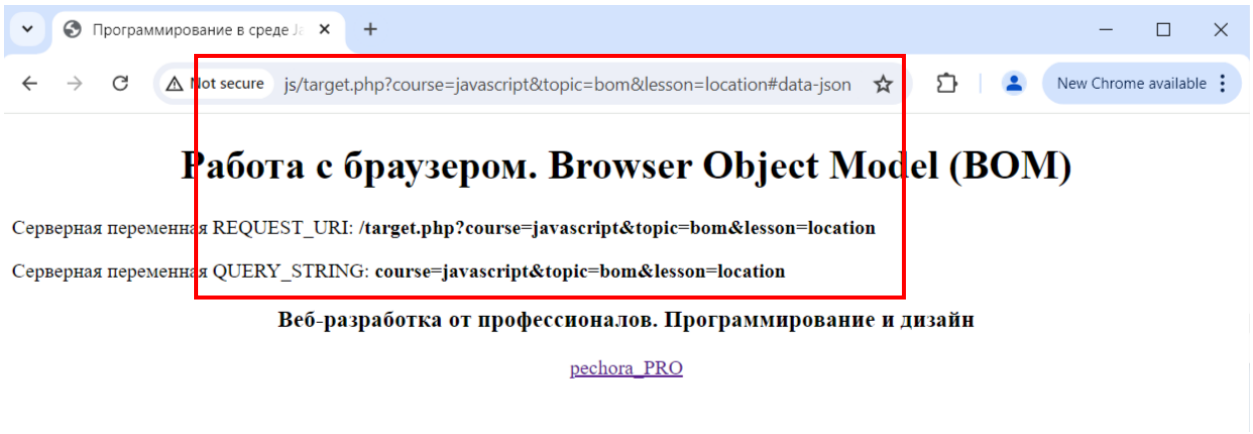
```
<?php

    echo 'Серверная переменная REQUEST_URI: <b>' .
    $_SERVER['REQUEST_URI'] . '</b><p>';

    echo 'Серверная переменная QUERY_STRING: <b>' .
    $_SERVER['QUERY_STRING'] . '</b>';

?>
```

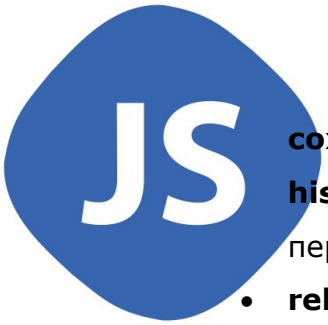
Идентификатор фрагмента (hash) присутствует в **строке запроса**, но не в **серверных** переменных.



Методы объекта

Объект **location** предоставляет ряд **методов**, которые можно использовать для управления адресом веб-страницы:

- **assign**(url) – **загружает** ресурс, который находится по пути **url**;
- **replace**(url) – **заменяет** текущую веб-станицу **другим ресурсом**, который находится по пути **url**. В отличие от метода **assign**, который также загружает веб-станицу с другого ресурса, метод **replace не**



- **сохраняет предыдущую веб-страницу в стеке истории переходов history**, поэтому мы **не сможем** вызвать метод **history.back**() для перехода к ней;
- **reload** (forcedReload) – перезагружает текущую веб-страницу. Параметр forcedReload указывает, надо ли использовать кэш браузера. Если параметр равен **true**, то кэш не используется.

Для перенаправления на другой ресурс мы можем использовать как **свойства**, так и **методы location**.

example_7.

```
Свойство Href: <button id="btn-href">Go.Href</button><p>
Метод Assign: <button id="btn-assign">Go.Assign</button><p>
<!-- обратите внимание, шаг назад это не предыдущая страница -->
Метод Replace: <button id="btn-replace">Go.Replace</button><p>
Метод Reload: <button id="btn-reload">Go.Reload</button>
<script>
    // свойство href
    document.getElementById("btn-href").onclick = () => {
        location.href = "target.html";
    }
    // метод assign
    document.getElementById("btn-assign").onclick = () => {
        location.assign("target.html");
    }
    // метод replace
    document.getElementById("btn-replace").onclick = () => {
        location.replace("target.html");
    }
```



```
    // метод reload
    document.getElementById("btn-reload").onclick = () => {
        location.reload(true);
    }
</script>
```

example_7. target.html

```
<button id="btn" onclick="history.back()">Back</button><p>
<img src="nu-pogodi.jpg" width="600px">
```

Передача данных в строке запроса

При работе с веб-приложениями на JavaScript часто требуется **передача данных через URL**. Однако не все символы можно безопасно использовать в строке URL, поэтому они должны быть **закодированы** в формате, который понимает URL.

Существует несколько способов кодировки данных строки запроса:

- Кодировка **urlencoded** — стандартное кодирование URL. Все символы, кроме английских букв, цифр и - _ . ! ~ * ' ( ), заменяются на их цифровой код в UTF-8 со знаком %.
- Кодировка **multipart/form-data** — используется для пересылки файлов. В этой кодировке поля пересылаются одно за другим через строку-разделитель.
- Кодировка **UTF-8** — используется всегда, независимо от языка и кодировки страницы. Если сервер ожидает данные в другой кодировке, например, windows-1251, то их нужно будет перекодировать.

Вопрос кодировки данных чрезвычайно важен (и объемен) и будет рассмотрен в соответствующих уроках.

Примечание. Вопросами кодировки данных строки запроса займемся в уроке **Объект URL**.

Передача простых данных

Передадим между клиентскими сценариями простые **текстовые** параметры.

example_8.

```
<a href=»/target.html?topic=Управляющие
конструкции&lesson=Специальные циклы&section=Цикл for-of»>
    Переход к файлу target.html
</a>
```

Внимательно изучайте код принимающего файла **target.html**. Логика обработки принимаемых данных будет усложняться.

example_8. **target.html**

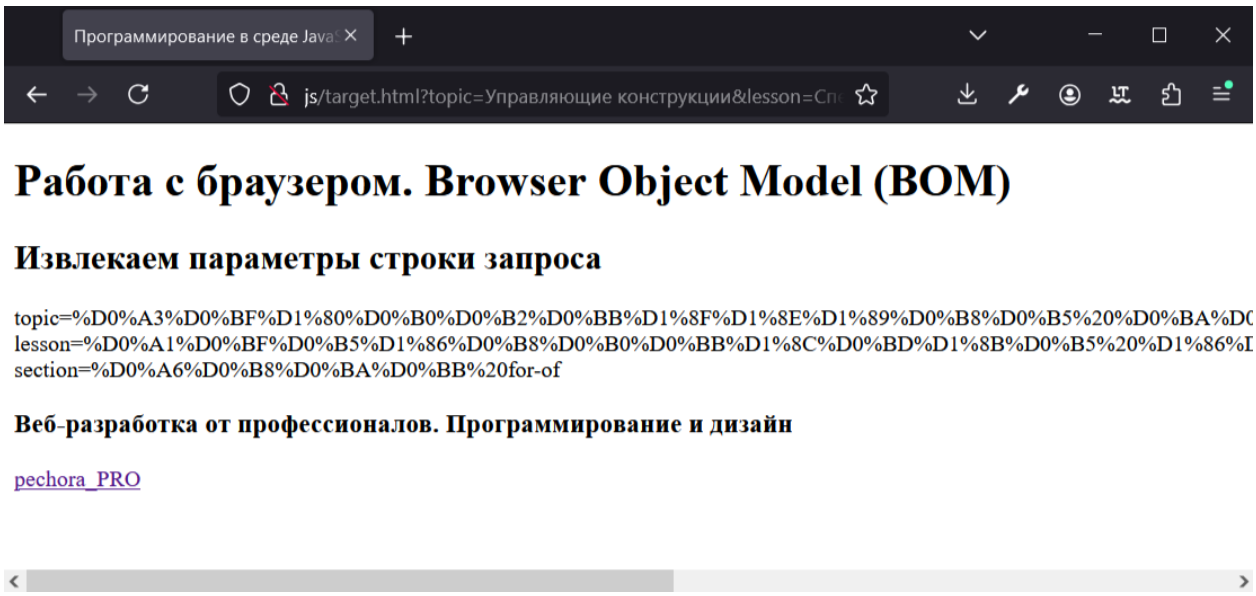
```
<script>
    // извлекаем строку запроса
    raw_search = location.search;
    // удаляем символ ?
    search = raw_search.substr(1);
    // извлекаем параметры строки запроса
    [topic, lesson, section] = search.split("&");
    // выводим параметры в браузер
    document.write(topic, '<br>');
    document.write(lesson, '<br>');
    document.write(section, '<br>');
```

```
</script>
```

На скриншоте представлен вывод в браузер **параметров** строки запроса в **закодированном** виде. Браузер автоматически кодирует URL-данные.

> ***Важно***. Браузер **автоматически** кодирует данные строки запроса.

Несмотря на то, что в адресной строке отображается текст кириллицей, отправка данных на сервер будет выполнена в закодированном виде.



Проведем обработку данных и выведем в браузер не сами параметры строки запроса, а их значения.

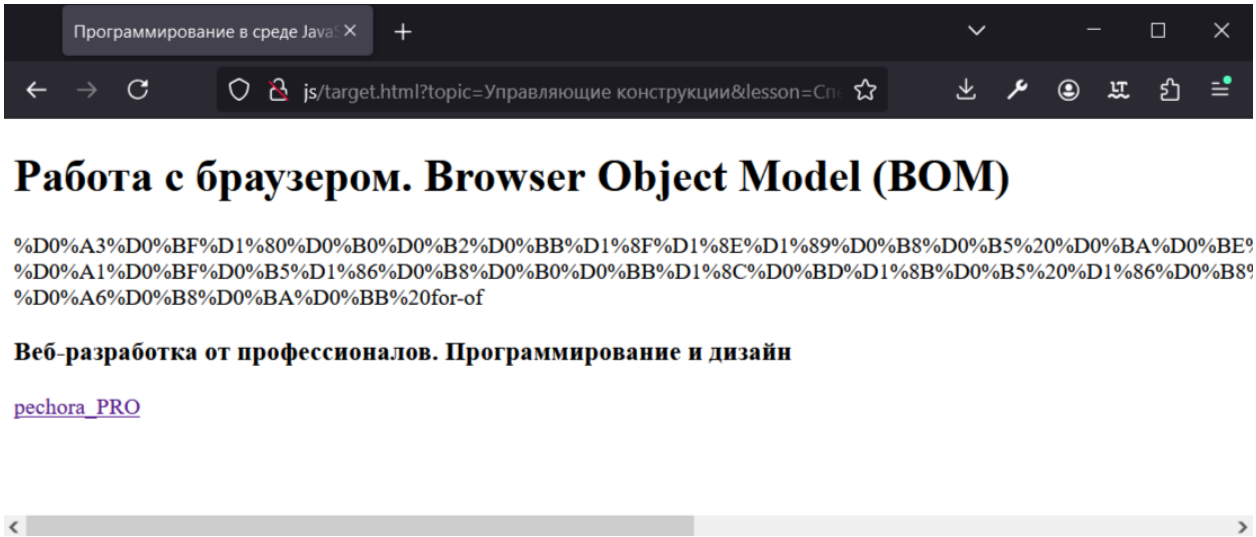
example_9.

```
<a href=»/target.html?topic=Управляющие
конструкции&lesson=Специальные циклы&section=Цикл for-of»>
    Переход к файлу target.html
</a>
```

example_9. target.html

```
<script>
    // извлекаем параметры строки запроса
    [topic, lesson, section]  =
    location.search.substr(1).split('&');
    // извлекаем значения параметров строки запроса
    [, topic_val] = topic.split(«=»);
    [, lesson_val] = lesson.split(“=”);
    [, section_val] = section.split(“=”);
    // выводим значения в браузер
    document.write(topic_val, '<br>');
    document.write(lesson_val, ‘<br>’);
    document.write(section_val, ‘<br>’);
</script>
```

Значения параметров в браузере.



В демонстрационном примере **example_10** подключим к обработке данных адресной строки цикл **for**.

example_10.

```
<a href="/target.html?topic=Управляющие
конструкции&lesson=Специальные циклы&section=Цикл for-of">
    Переход к файлу target.html
</a>
```

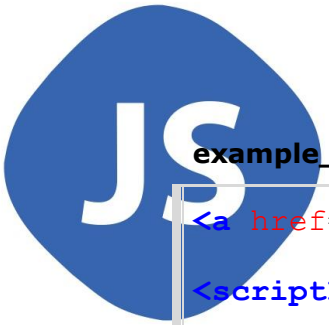
Вот теперь код обработки строки запроса выглядит почти профессионально (значения передаваемых параметров, все-таки, пока не читаемы).

example_10. **target.html**

```
<script>
    // получаем массив параметров
    search = location.search.substr(1).split('&');
    arr = new Array();
    // сохраняем значения параметров в массив
    for (par of search) {
        arr.push(par.split("=")[1]);
    }
    // выводим значения в браузер
    document.write(arr[0], '<br>');
    document.write(arr[1], '<br>');
    document.write(arr[2], '<br>');
    // console.log(arr);
</script>
```

Передача данных формата JSON

Если передать в строке запроса можно простую строку, то можно и строку **формата JSON**.



example_11.

```
<a href="#" id="link">Передача строки в формате JSON</a>
<script>
    // формируем ссылку с данными формата JSON
    let href = '/target.html?par=' + JSON.stringify(
        {
            topic: "Введение в JavaScript",
            lesson: [
                "Введение в сценарии JS",
                "Переменные и константы",
                "Типы данных"
            ]
        }
    );
    // вставка ссылки с данными в атрибут href
    document.getElementById('link').setAttribute('href', href);
</script>
```

example_11. target.html

```
<script>
    // извлекаем JSON-значение из параметра
    let strJSON = location.search.substr(1).split('=')[1];
    document.write(strJSON);
    console.log(strJSON);
    // попытка распарсить JSON-строку приведет к ошибке
    arr = JSON.parse(strJSON); // Uncaught SyntaxError
```

```
</script>
```

Попытка распарсить JSON-строку приведет к синтаксической ошибке (нечитаемые символы при преобразовании строки JSON).

Проведем небольшой эксперимент и выполним передачу строки, состоящей из символов латиницы.

example_12.

```
<a href="#" id="link">Передача строки в формате JSON</a>
<script>
    // формируем ссылку с данными формата JSON
    let href = '/target.html?par=' + JSON.stringify(
        {
            topic: "Vvedenie-v-JavaScript",
            lesson: [
                "Vvedenie-v-scenarii-JS",
                "Peremennye-i-konstanty",
                "Tipy-dannyh"
            ]
        }
    );
    // вставка ссылки с данными в атрибут href
    document.getElementById('link').setAttribute('href', href);
</script>
```

example_12. **target.html**

```
<script>
```

```
    // извлекаем JSON-значение из параметра
    let strJSON = location.search.substr(1).split('=')[1];
    document.write(strJSON);
    console.log(strJSON);
    // попытка распарсить JSON-строку приведет к ошибке
    arr = JSON.parse(strJSON); // Uncaught SyntaxError
</script>
```

Вывод. Вывод простой. Передаваемые данные необходимо кодировать / декодировать. **Почти всегда**.

Почти всегда. Как определить это самое *почти*...

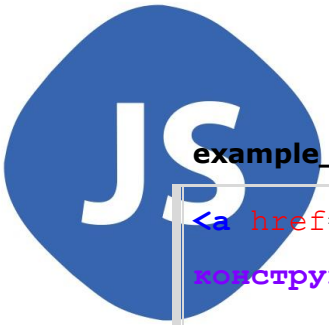
Далее приведу пример, чрезвычайно важный для уяснения сути различий между **front** и **back** разработкой.

Передача данных файлу **target.html** – требует декодирования передаваемых данных. Передача данных файлу **target.php** – нет. Все дело в способе обработки данных строки запроса.

- При использовании в качестве целевого ресурса **HTML-файла**, обработкой строки запроса занимается **JS-движок** в **JS-скрипте** запрашиваемого файла.
- При использовании в качестве целевого ресурса **PHP-файла**, обработкой строки запроса занимается **PHP-интерпретатор** в **PHP-скрипте** запрашиваемого файла.

Важно. Вот!

А теперь совместим обработку данных в **серверном** и **клиентском** сценариях одного файла. Никто не мешает нам это сделать.



example_13.

```
<a href="/target.php?topic=Управляющие
конструкции&lesson=Специальные циклы&section=Цикл for-of">
    Переход к файлу target.php
</a>
```

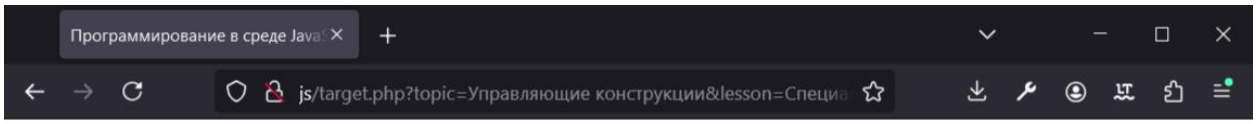
example_13. target.php

```
<h2>Работает PHP!</h2>
<!-- этот сценарий выполнится на сервере PHP-движком -->
<?php
    // выводим значения в браузер
    echo <<<HERE
        Тема: {$_GET["topic"]}<br>
        Урок: {$_GET["lesson"]}<br>
        Раздел: {$_GET["section"]}
    HERE;
?>
<h2>Работает JS!</h2>
<!-- этот сценарий выполнится на клиенте JS-движком -->
<script>
    // получаем массив параметров
    search = location.search.substr(1).split('&');
    arr = new Array();
    // сохраняем значения параметров в массив
    for (par of search) {
        arr.push(par.split("=")[1]);
```



```
    }
    // выводим значения в браузер
    document.write(`
        Тема: ${arr[0]}<br>
        Урок: ${arr[1]}<br>
        Раздел: ${arr[2]}
    `);
</script>
<h2>Работает Браузер (построение DOM)!</h2>
```

Следующий скриншот запечатлел полет моей *гениальной* идеи растолковать что, к чему... ;)



Работа с браузером. Browser Object Model (BOM)

Работает PHP!

Тема: Управляющие конструкции
Урок: Специальные циклы
Раздел: Цикл for-of

Работает JS!

Тема:
%D0%A3%D0%BF%D1%80%D0%B0%D0%B2%D0%BB%D1%8F%D1%8E%D1%89%D0%B8%D0%B5%20%D0%BA%D0%BE%D0%
Урок:
%D0%A1%D0%BF%D0%B5%D1%86%D0%B8%D0%B0%D0%BB%D1%8C%D0%BD%D1%8B%D0%B5%20%D1%86%D0%B8%D0%
Раздел: %D0%A6%D0%B8%D0%BA%D0%BB%20for-of

Работает Браузер (построение DOM)!

Веб-разработка от профессионалов. Программирование и дизайн

pechora_PRO

Никто не работает ;)

Впрочем, серверные сценарии – это уже другая история. Все вопросы к **backend**-разработчикам ;)



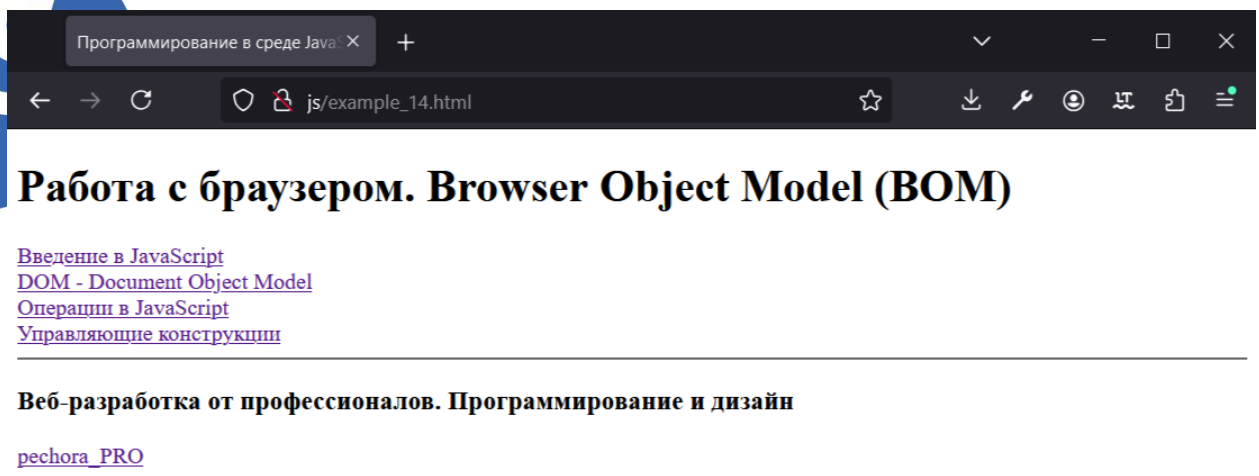
Практика построения навигационного меню

В следующих примерах приведу возможные способы организации навигационного меню средствами **клиентского** сценария.

Пример **одноуровневого** меню.

example_14.

```
<script src="courses.js"></script>
<script>
    // начальное значение переменной
    menu = '';
    // в цикле формируем меню
    courses.forEach( course => {
        menu += `
            <a href="target.php?topic=${course.topic}">
            ${course.topic}
            </a><br>
        `;
    } );
    // вывод меню в браузер
    document.write('<div id="menu">' + menu + '</div><hr>');
</script>
```



В качестве целевого файла для перехода буду использовать файл серверного сценария.

example_14. target.php

```php
<?php

    // серверный вывод данных массива GET

    echo "<pre>";

    print_r($_GET);

    echo "</pre>";

?>
```

example_14. courses.js

```javascript
let courses = [

    {

        id: 1,

        topic: "Введение в JavaScript"

    },

    // ...

]
```



Обработкой данных не занимаемся, оставим эти труды backend-коллеге.

> ***Примечание***. Один из вариантов использования полученных данных – организация **запросов к базе данных**.

В демонстрационном примере **example_15** перейдем к **динамическому** формированию ссылок меню. Будьте внимательны, дальше будет сложнее.

example_15.

```
<script src="courses.js"></script>
<script>
    // начальное значение переменной
    menu = '';
    // в цикле формируем меню
    courses.forEach( course => {
        menu += `
            <a href="#" id="topic-${course.id}">
            ${course.topic}
            </a><br>
        `;
    } )
    // вывод меню в браузер
    document.write('<div id="menu">' + menu + '</div><hr>')
    // выбор ссылок меню
    links = document.querySelectorAll("div#menu a");
    // навешиваем обработчик события click на ссылки меню
    for (link of links) {
        link.addEventListener("click", function() {
```

```
            let topic = this.innerText;
            let href = `target.php?topic=${topic}`;
            // выполняем переход на ресурс target.php
            location.assign(href);
        })
    }
</script>
```

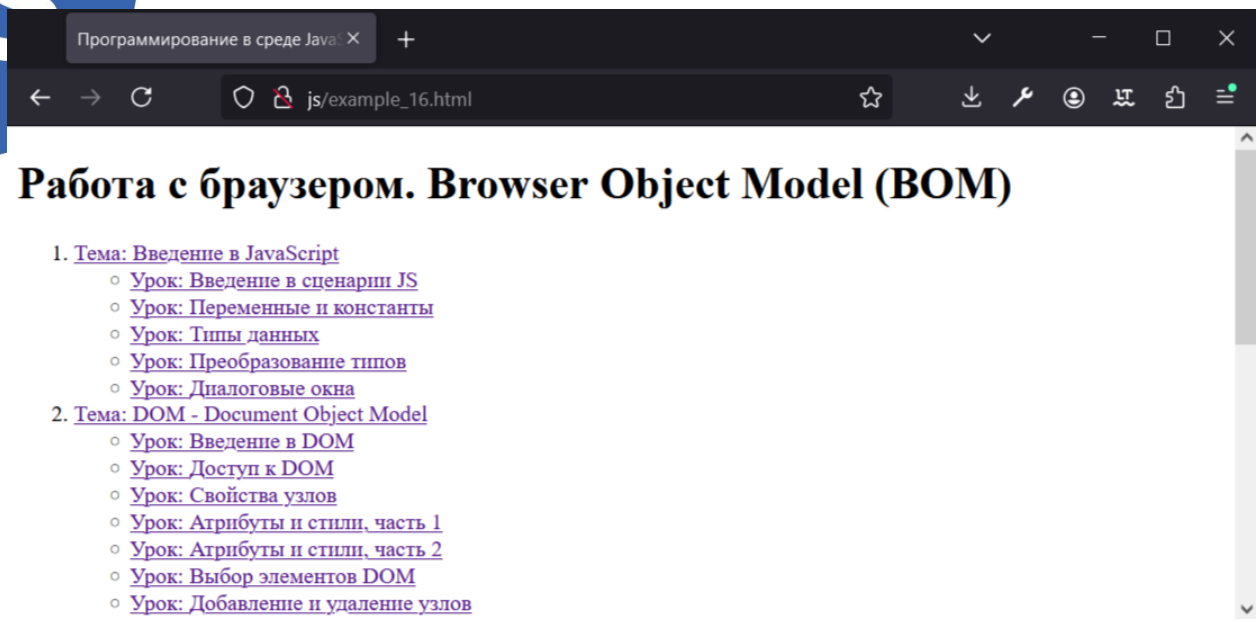
example_15. target.php

```
<?php
    // серверный вывод данных массива GET
    echo "<pre>";
    print_r($_GET);
    echo "</pre>";
?>
```

example_15. courses.js

```
let courses = [
    {
        id: 1,
        topic: "Введение в JavaScript"
    },
    // ...
]
```

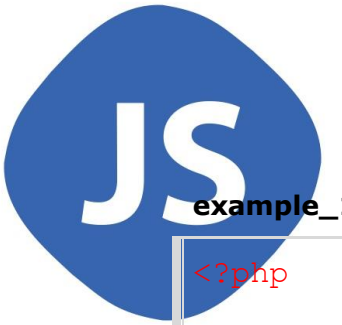
В демонстрационном примере **example_16** построим **двухуровневое** меню с динамическим назначением ссылок.



example_16.

```
<script src="courses.js"></script>
<script>
    // начальное значение переменной
    let menu = '';
    // во внешнем цикле формируем меню первого уровня
    for(course of courses) {
        menu += `
        <li id="topic-${course.id}">
        <a href="#">Тема: <span
        class="topic">${course.topic}</span></a>
        `;
        // во вложенном цикле формируем меню второго уровня
        submenu = '';
        for (lesson of course.lesson) {
            submenu += `<li><a href="#"">Урок: <span
```

```
            class="lesson">${lesson}</span></a></li>`;
        }
        menu += `<ul>${submenu}</ul></li>`;
    }
    // вывод меню в браузер
    document.write('<ol>' + menu + '</ol>');
    // выбор пунктов меню
    lis = document.querySelectorAll("ol > li");
    for (let li of lis) {
        // навешиваем обработчик события click
        li.addEventListener("click", function() {
            let topic = li.querySelector(".topic").innerText;
            // если клик по меню первого уровня
            if (event.target.className == "topic"){
                link = `topic=${topic}`;
            }
            // если клик по меню второго уровня
            if (event.target.className == "lesson"){
                lesson = event.target.innerText;
                link = `topic=${topic}&lesson=${lesson}`;
            }
            // выполняем переход на ресурс target.php
            location.href = "target.php?" + link;
        })
    }
</script>
```



example_16. target.php

```php
<?php

    // серверный вывод данных массива GET

    echo "<pre>";

    print_r($_GET);

    echo "</pre>";

?>
```

example_16. courses.js

```
let courses = [

    {

        id: 1,

        topic: "Введение в JavaScript",

        lesson: [

            "Введение в сценарии JS",

            "Переменные и константы",

            "Типы данных",

            "Преобразование типов",

            "Диалоговые окна"

        ]

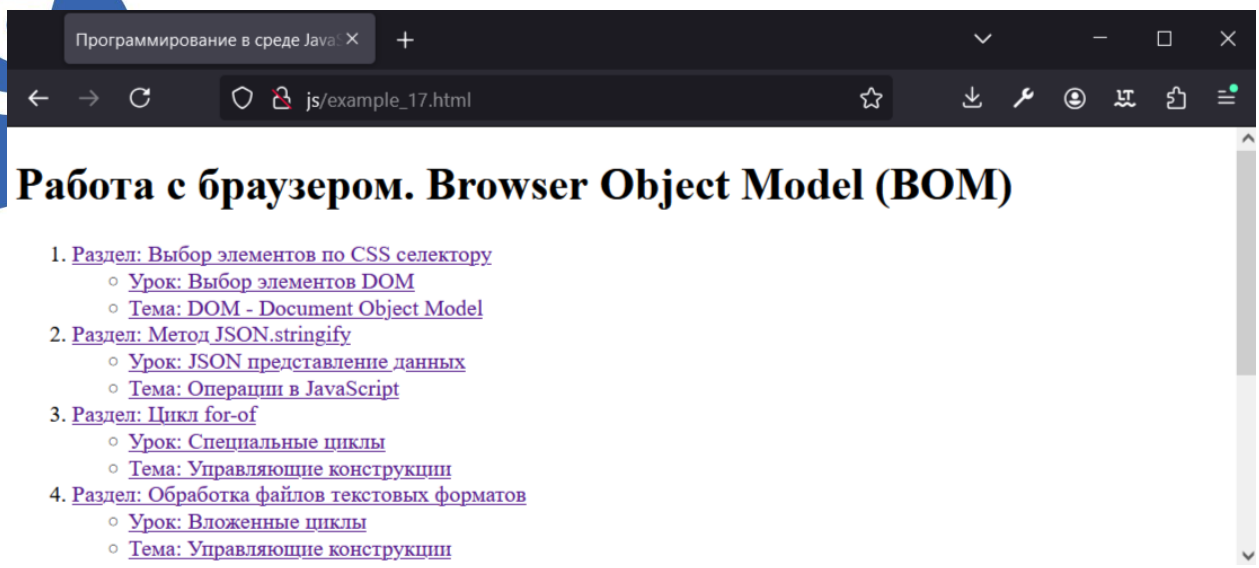
    },

    // ...

]
```

В качестве последнего примера урока рассмотрим еще один вариант меню. Допустим, вам необходимо вывести наиболее значимые (интересные / обсуждаемые / и др.) разделы урока.





При этом необходимо понимать, выбор **раздела** урока потребует отправки данных:

- Раздел
- Урок
- Тема

Выбор **урока** темы потребует отправки данных:

- Тема
- Урок

Выбор **темы** курса потребует отправки данных: **Тема**.

example_17.

```
<script>

    // начальное значение переменной

    let menu = '';

    // формируем пункты меню

    for(item of sections) {

        menu += `

            <li>

                <a href="#">Раздел: <span
```

```
                class="section">${item.section}</span></a>
                <ul>
                    <li><a href="#"">Урок: <span
                    class="lesson">${item.lesson}</span></a></li>
                    <li><a href="#">Тема: <span
                    class="topic">${item.topic}</span></a></li>
                </ul>
            </li>
        `;
}
// вывод меню в браузер
document.write('<ol>' + menu + '</ol>');
// выбор пунктов меню
lis = document.querySelectorAll("ol > li");
// динамическое навешивание событий на ссылки меню
for (let li of lis) {
    li.addEventListener("click", function() {
        let section = li.querySelector(".section").innerText;
        let lesson = li.querySelector(".lesson").innerText;
        let topic = li.querySelector(".topic").innerText;
        if (event.target.className == "topic"){
            link = `topic=${topic}`;
        }
        if (event.target.className == "lesson"){
            link = `topic=${topic}&lesson=${lesson}`;
        }
        if (event.target.className == "section") {
```

```
                link =
                `topic=${topic}&lesson=${lesson}&section=${sectio
                n}`;
            }
            // выполняем переход на ресурс target.php
            location.assign("target.php?" + link);
        })
    }
</script>
```

example_17. target.php

```
<?php
    // серверный вывод данных массива GET
    echo "<pre>";
    print_r($_GET);
    echo "</pre>";
?>
```

example_17. sections.php

```
let sections = [
    {
        section: "Выбор элементов по CSS селектору",
        lesson: "Выбор элементов DOM",
        topic: "DOM - Document Object Model"
    },
    // ...
```



Задача backend-разработчика **принять** пересылаемые данные и выполнить **запрос** к базе данных на получение запрашиваемых ресурсов.

Больше примеров и практики в **самостоятельных работах**:

- Создание меню сайта из предложенного массива.
- Создание динамических ссылок для элементов меню.
- Создание ссылок с использованием параметров адресной строки.
- Ревьюирование клиентского сценария вывода меню.

P.S.

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой **теме** курса:

- **Практические работы**.

К **курсу**:

- **Дополнительный практикум.**

